Dinamikus programozás

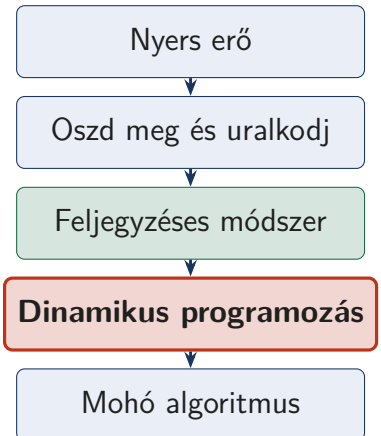
és a 0-1 hátizsákprobléma

Nyers erő · Oszd meg és uralkodj · Feljegyzés · Mohó

Tartalom

1. A hátizsákprobléma
2. Nyers erő
3. Oszd meg és uralkodj
4. Feljegyzéses módszer
5. Dinamikus programozás
6. A példa végigszámolva
7. A megoldás előállítása
8. További DP-feladatok
9. Mohó algoritmus
10. Elméleti háttér

Öt stratégia ugyanarra a feladatra



$\mathcal{O}(2^N)$ — minden esetet kipróbál

rekurzió, de sok az ismétlés

rekurzió + részeredmény-tár

alulról felfelé, táblázat

gyors, de nem mindig optimális

A 0-1 hátizsákprobléma

Bemenet

- $W_{\max}$ — a hátizsák mérete
- N — a rendelkezésre álló tárgyak száma
- w_i — az i. tárgy súlya, p_i — az i. tárgy értéke

Kimenet

$P_{\max}$ — egy optimális pakolás értéke

A név: minden tárgyból 0 vagy 1 darabot vihetünk — nem lehet feldarabolni.

Alapfogalmak

- Pakolás: tárgyanként meghatározzuk, hogy bekerül-e a zsákba vagy sem (pl. N elemű logikai tömb).
- Összsúly / összérték: a pakolás által a hátizsákba választott tárgyak súlyának / értékének összege.
- Érvényes pakolás: amelynek összsúlya nem nagyobb, mint a hátizsák mérete.
- Optimális pakolás: olyan érvényes pakolás, amelyiknél nincs nagyobb összértékű érvényes pakolás.

A futó példánk

i	1	2	3	4	5	6
w_i	2	1	1	1	3	2
p_i	4	3	2	8	7	5

$N = 6$ tárgy, a hátizsák mérete $W_{\max} = 4$.

Melyik tárgyakat vigyük el?

Nyers erő módszere (brute force)

Alapelv

Egyszerűen végigpróbálgatjuk az összes lehetséges megoldást.

- Minden tárgyra két lehetőség van: bekerül vagy nem $\Longrightarrow$ 2^N pakolás.
- Amennyiben érvényes pakolást találunk, ellenőrizzük, hogy jobb-e, mint az eddig talált legjobb.
- Ha igen, akkor ezt követően ezt tekintjük a legjobbnak.

Nyers erő — pszeudokód

```
függvény HátizsákBF
    OPT ← [hamis, hamis, ..., hamis]
    ciklus i ← 1-től 2^N-1-ig
        ciklus j ← 0-tól N-1-ig
            K[j] ← ⌊i / 2^j⌋ mod 2 = 1
        ciklus vége
        ha (ÖsszSúly(K)≤W_max) ∧ (ÖsszÉrték(K)>ÖsszÉrték(OPT)) akkor
            OPT ← K
        elágazás vége
    ciklus vége
    vissza ÖsszÉrték(OPT)
függvény vége
```

A nyers erő értékelése

Előnyei

- Egyszerűen megérthető
- Egyszerűen implementálható
- Gyakran csak ez használható
- Tárigénye nem jelentős
- Jól párhuzamosítható

Hátránya

- Futásideje nem kedvező: 2^N lépés

N	2^N
20	1 048 576
40	$1{,}1 \cdot 10^{12}$
60	$1{,}2 \cdot 10^{18}$

Oszd meg és uralkodj

Ismerős módszer: összefésüléses rendezés, gyorsrendezés, k-adik legkisebb elem kiválasztása.

Az alapelv (rekurzív)

- Ha a probléma egyszerű, oldjuk meg közvetlenül.
- Különben bontsuk fel kisebb részproblémákra, oldjuk meg azokat (tipikusan rekurzív hívással), és a részeredményekből állítsuk elő a megoldást.

Ez az alapelv optimalizációra is használható.

Mi egy részprobléma?

Egy részprobléma esetén van

- t darab tárgyunk
  (az eredeti feladat $1..t$ indexű tárgyai)
- h szabad helyünk
  (ez értelemszerűen nem nagyobb, mint $W_{\max}$)

Jelölje $F_{t,h}$ az ilyen részprobléma optimális pakolásának értékét.

A rekurzív megfogalmazás

$$F_{t,h} = \begin{cases} 0 & \text{ha } h = 0 \\ 0 & \text{ha } t = 0 \\ F_{t-1,h} & \text{ha } h > 0 \wedge t > 0 \wedge h < w_t \\ \max\left\{F_{t-1,h},\ F_{t-1,h-w_t} + p_t\right\} & \text{ha } h > 0 \wedge t > 0 \wedge h \geq w_t \end{cases}$$

A teljes feladat megoldása: $F_{N,W_{\max}}$

A képlet szavakban

- Ha a tárgyak száma vagy a szabad hely 0, akkor az optimális pakolás értéke 0.
- Ha az utolsó tárgy nem fér bele, akkor az érték annyi, mint amennyi az előtte lévő tárgyakkal elérhető ugyanekkora helyen.
- Ha befér, akkor a kettő közül a nagyobbik:
  - *nem rakjuk be:* az előtte lévő tárgyakkal elérhető optimum ugyanekkora helyen;
  - *berakjuk:* az előtte lévőkkel elérhető érték w_t-vel csökkentett helyen, plusz p_t.

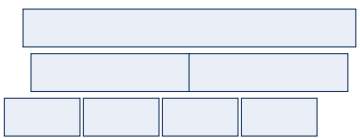
Oszd meg és uralkodj — pszeudokód

```
függvény LegjobbRészMegoldás(t, h)
    ha (t = 0) ∨ (h = 0) akkor
        vissza 0
    különben
        P_nem ← LegjobbRészMegoldás(t-1, h)
        ha h ≥ w_t akkor
            P_igen ← p_t + LegjobbRészMegoldás(t-1, h-w_t)
            P_opt ← max(P_igen, P_nem)
        különben
            P_opt ← P_nem
        elágazás vége
        vissza P_opt
    elágazás vége
függvény vége
```

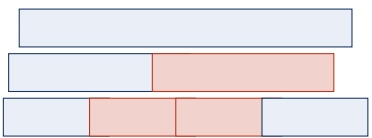
A teljes feladat: `LegjobbRészMegoldás(N,` $W_{\max}$`)`

A baj: átfedő részproblémák

Így képzeljük:



A valóságban:



- A felbontásnál nem foglalkoztunk azzal, hogy ugyanazt a részproblémát többször is megkaphatjuk.
- Így ugyanazt újra és újra megoldjuk — lásd a Fibonacci-számok rekurzív számítását.

Feljegyzéses módszer (memoization)

Az ötlet

A futásidő egy egyszerű módszerrel javítható:

- a már kiszámított részeredményeket eltároljuk;
- ha egy már megoldott részproblémához jutunk újra, akkor felhasználjuk a tárolt értéket.

A megoldás alapelve pontosan ugyanaz, mint az „oszd meg és uralkodj” esetében.

Feljegyzéses módszer — pszeudokód

```
függvény LegjobbRészMegoldás(t, h)
    ha (t = 0) ∨ (h = 0) akkor vissza 0

    ha TárolóbanKeres([t, h]) ≠ ∅ akkor
        vissza TárolóbanKeres([t, h])

    ... innen változatlan: P_nem, P_igen, P_opt kiszámítása ...

    TárolóbaFeljegyez([t, h], P_opt)
    vissza P_opt
függvény vége
```

A piros sorok az egyetlen különbség az előző algoritmushoz képest.

A feljegyzéses módszer értékelése

Előnyei

- Jelentősen csökkenti a lépésszámot, ha sok egymást átfedő részproblémát kell megoldani.
- Egyszerűen implementálható, általános séma adható rá.

Hátrányai

- Csak bizonyos szerkezetű megoldásoknál hatékony.
- Jelentős többlet tárhelyigényt jelent.

Tulajdonképpen jobb futásidőt kapunk nagyobb tárhelyért cserébe.

Fordítsuk meg az irányt!

- Az „oszd meg és uralkodj” felülről lefelé halad: a rekurzív függvény először csak hívogatja magát, amíg el nem jut valamelyik triviális megoldásig.
- A feljegyzéses módszernél jól látszik, hogy a tárolóba először a legegyszerűbb részproblémák adatai kerülnek bele, és ezekből épülnek fel a komplexebbek.
- Próbáljuk meg eleve ezt az alulról felfelé való építkezést megvalósítani!

Ez a dinamikus programozás.

Honnan a név?

Richard Bellman, RAND Corporation, 1950

A módszer a nevét nem a tartalmáról kapta. Bellman visszaemlékezése szerint a védelmi miniszter „*kórosan félt és utálta*" a *research* és a *mathematical* szavakat, ezért a kutatásának álcázó nevet keresett:

- *programming* — akkoriban „tervezés, ütemezés" (mint a *lineáris programozásban*);
- *dynamic* — fizikai jelentése miatt lehetetlen volt pejoratívan használni.

A módszer lényege tehát: táblázatkitöltés, nem programozás.

Mikor használható a módszer?

A hatékony használat két előkövetelménye

1. Optimális részstruktúra: a feladat optimális megoldása önmagán belül a részfeladatok optimális megoldásait is tartalmazza.
2. Átfedő részfeladatok: a részfeladatokra bontás során többször is felmerül ugyanannak a részfeladatnak a megoldása.

Ha mindkettő teljesül, a feladat egyszerűen megoldható.

A megoldás négy lépése

1. Az optimális megoldás szerkezetének jellemzése.
2. A megoldás értékének (jóságának) rekurzív definiálása.
3. Az optimális érték kiszámítása alulról felfelé történő módon.
4. Amennyiben szükséges, magának az optimális megoldásnak a megadása.

A 4. lépés csak akkor kell, ha nem elég az optimum *értéke*, hanem magát a megoldást is tudni szeretnénk.

A hátizsák DP-táblája

- Az algoritmus egy F tömböt tölt fel, mérete $(N+1) \times (W_{\text{max}}+1)$.
- $F[t, h]$ azt mutatja, hogy az első t darab tárgy elhelyezése h szabad helyre milyen optimális pakolási összértékkel járhat.
- A triviális esetek előre beírhatók:
  - ha nincs szabad hely ($h = 0$), az érték 0;
  - ha nincs tárgy ($t = 0$), az érték 0.
- A teljes feladat megoldása: $F[N, W_{\text{max}}]$.

Miért nincs szükség rekurzióra?

A rekurzív képlet az $F[t, h]$ elem kiszámításához csak a t-nél és h-nál kisebb indexű elemeket igényli.

- Ezért ha a t-t és h-t növekvő sorrendben járjuk be, a szükséges részeredmények mindig már rendelkezésre állnak.
- Így a feltöltés egyszerű két egymásba ágyazott ciklussal megoldható.

Pszeudokód — rekurzió nélkül

```
függvény HátizsákDP                    % a tömbök 0-tól indexelve
    % triviális esetek:  F[t,0] ← 0   és   F[0,h] ← 0
    ciklus t ← 1-től N-ig
        ciklus h ← 1-től W_max-ig
            ha h ≥ w_t akkor
                F[t,h] ← max(F[t-1,h], F[t-1,h-w_t] + p_t)
            különben
                F[t,h] ← F[t-1,h]
            elágazás vége
        ciklus vége
    ciklus vége
    vissza F[N,W_max]
függvény vége
```

Lépésszám és tárigény

A tábla $(N+1)(W_{\max}+1)$ cellából áll,

és egy cella kitöltése konstans idő.

lépésszám: $\mathcal{O}(N \cdot W_{\max})$ tárigény: $\mathcal{O}(N \cdot W_{\max})$

Összevetésül $N = 60$, $W_{\max} = 10\,000$ esetén:
nyers erő $\approx 1{,}2 \cdot 10^{18}$ vs. DP $= 600\,000$ lépés.

A feladat

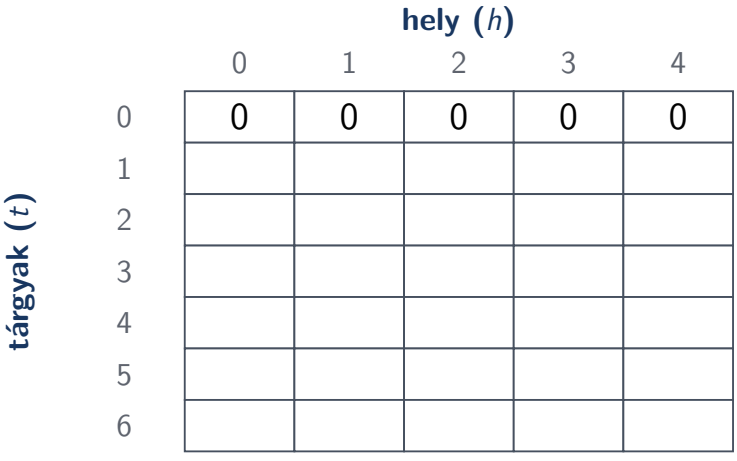
i	1	2	3	4	5	6
w_i	2	1	1	1	3	2
p_i	4	3	2	8	7	5

$N = 6, \quad W_{\max} = 4$

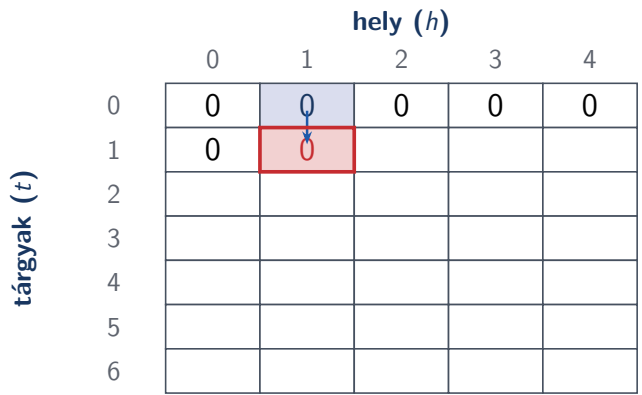
A tábla

- 7 sor ($t = 0 \dots 6$)
- 5 oszlop ($h = 0 \dots 4$)
- soronként, balról jobbra töltjük

Inicializálás: a 0. sor és a 0. oszlop nulla



1. eset: a tárgy nem fér el ($h < w_t$)



$t = 1,\ h = 1$
$w_1 = 2,\ p_1 = 4$

Mivel $h < w_t$, a 2 súlyú tárgy nem fér az 1 méretű zsákba.

Másoljuk át a felette lévő mezőt.

2. eset: befér és érdemes berakni

tárgyak (t) \ hely (h)	0	1	2	3	4
0	0	0	0	0	0
1	0	0	4	4	4
2	0	3	4	7	7
3	0	3	5	7	9
4	0	8	11		
5					
6					

$t = 4,\ h = 2$

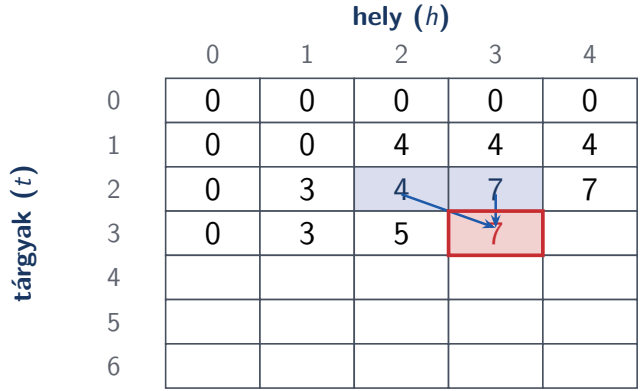
$w_4 = 1,\ p_4 = 8$

nélküle: $F[3,2] = 5$

vele: $F[3,1] + p_4 = 3 + 8 = 11$

$\max\{5, 11\} = 11$

3. eset: befér, de nem érdemes



$$t = 3,\ h = 3$$
$$w_3 = 1,\ p_3 = 2$$

nélküle: $F[2,3] = 7$

vele: $F[2,2] + p_3 = 4 + 2 = 6$

$\max\{7, 6\} = 7$

A kitöltés: $t = 1$ (első tárgy, $w = 2$, $p = 4$)

tárgyak (t) \ hely (h)	0	1	2	3	4
0	0	0	0	0	0
1	0	0	4	4	4
2					
3					
4					
5					
6					

$t = 2 \quad (w = 1,\ p = 3)$

hely (h)

tárgyak (t)	0	1	2	3	4
0	0	0	0	0	0
1	0	0	4	4	4
2	0	3	4	7	7
3					
4					
5					
6					

$t = 3 \quad (w = 1,\ p = 2)$

hely (h)

tárgyak (t)	0	1	2	3	4
0	0	0	0	0	0
1	0	0	4	4	4
2	0	3	4	7	7
3	0	3	5	7	9
4					
5					
6					

$t = 4 \quad (w = 1,\ p = 8)$

hely (h)

tárgyak (t)	0	1	2	3	4
0	0	0	0	0	0
1	0	0	4	4	4
2	0	3	4	7	7
3	0	3	5	7	9
4	0	8	11	13	15
5					
6					

$t = 5 \quad (w = 3,\ p = 7)$

tárgyak (t) \ hely (h)	0	1	2	3	4
0	0	0	0	0	0
1	0	0	4	4	4
2	0	3	4	7	7
3	0	3	5	7	9
4	0	8	11	13	15
5	0	8	11	13	15
6					

$t = 6$: kész — az optimum $F[6, 4] = 16$

tárgyak (t) \ hely (h)	0	1	2	3	4
0	0	0	0	0	0
1	0	0	4	4	4
2	0	3	4	7	7
3	0	3	5	7	9
4	0	8	11	13	15
5	0	8	11	13	15
6	0	8	11	13	16

Melyik tárgyakat vigyük?

Eddig csak az optimum értékét tudjuk (16). A gyakorlatban azonban magára a megoldásra van szükség.

A felépített táblából ez is kiolvasható

Induljunk a $(t = N,\ h = W_{\max})$ cellától, és haladjunk visszafelé:

- ha $F[t, h] = F[t-1, h]$, akkor a t. tárgy nincs a zsákban;
- ha a két érték nem egyezik, akkor a t. tárgy benne van — ilyenkor h-t csökkentjük w_t-vel.

A visszafejtés pszeudokódja

```
függvény HátizsákDPEredmény(F)
    OPT ← [hamis, hamis, ..., hamis]
    t ← N
    h ← W_max
    ciklus amíg (t > 0) ∧ (h > 0)
        ha F[t,h] ≠ F[t-1,h] akkor
            OPT[t] ← igaz        % a t. tárgy benne van
            h ← h - w_t
        elágazás vége
        t ← t - 1
    ciklus vége
    vissza OPT
függvény vége
```

Visszafejtés: $t = 6,\ h = 4$

tárgyak (t) \ hely (h)	0	1	2	3	4
0	0	0	0	0	0
1	0	0	4	4	4
2	0	3	4	7	7
3	0	3	5	7	9
4	0	8	11	13	15
5	0	8	11	13	15
6	0	8	11	13	16

$F[6, 4] = 16$
$F[5, 4] = 15$
Eltérnek $\Rightarrow$ a 6. tárgy benne van.
$h \leftarrow 4 - w_6 = 2$

Visszafejtés: $t = 5,\ h = 2$

tárgyak (t) \ hely (h)	0	1	2	3	4
0	0	0	0	0	0
1	0	0	4	4	4
2	0	3	4	7	7
3	0	3	5	7	9
4	0	8	11	13	15
5	0	8	11	13	15
6	0	8	11	13	16

$F[5, 2] = 11$
$F[4, 2] = 11$

Egyeznek $\Rightarrow$ az 5. tárgy nincs benne.

h változatlan.

Visszafejtés: $t = 4,\ h = 2$

tárgyak (t) \ hely (h)	0	1	2	3	4
0	0	0	0	0	0
1	0	0	4	4	4
2	0	3	4	7	7
3	0	3	5	7	9
4	0	8	11	13	15
5	0	8	11	13	15
6	0	8	11	13	16

$F[4, 2] = 11$
$F[3, 2] = 5$
Eltérnek $\Rightarrow$ a 4. tárgy benne van.
$h \leftarrow 2 - w_4 = 1$

Visszafejtés: $t = 2,\ h = 1$

hely (h)

tárgyak (t)	0	1	2	3	4
0	0	0	0	0	0
1	0	0	4	4	4
2	0	3	4	7	7
3	0	3	5	7	9
4	0	8	11	13	15
5	0	8	11	13	15
6	0	8	11	13	16

($t = 3$: $F[3,1] = F[2,1]$, kimarad)
$F[2,1] = 3, \quad F[1,1] = 0$
Eltérnek $\Rightarrow$ a 2. tárgy benne van.
$h \leftarrow 1 - w_2 = 0 \quad \Rightarrow$ vége.

Az eredmény

Egy optimális pakolás

a 2., a 4. és a 6. tárgy

i	2	4	6	összesen
w_i	1	1	2	4 $(= W_{\max})$
p_i	3	8	5	16

A zsák pontosan tele van, az érték a lehető legnagyobb.

Tárigény: elég egyetlen sor

Ha csak az értékre van szükségünk, akkor mindig csak az előző sorra hivatkozunk:

$$F[t,h] = \max\Big\{F[t-1,h],\ F[t-1,h-w_t]+p_t\Big\}$$

Egysoros változat

Egyetlen $W_{\max}+1$ hosszú tömb elég, ha a h ciklust csökkenő sorrendben futtatjuk: $h = W_{\max},\ W_{\max}-1,\ \ldots,\ w_t$

tárigény: $\mathcal{O}(NW_{\max}) \longrightarrow \mathcal{O}(W_{\max})$

Miért csökkenő sorrendben?

- Az $F[h - w_t]$ cellának még az előző sorbeli (a t. tárgy nélküli) értékét kell tartalmaznia.
- Csökkenő h mellett a kisebb indexű cellákhoz még nem nyúltunk hozzá ebben a körben $\Rightarrow$ helyes.
- Növekvő h mellett a t. tárgyat többször is beraknánk — ez viszont már a *korlátlan* (*unbounded*) hátizsák feladata!

Ugyanaz a kód, egyetlen ciklusirány dönti el, melyik feladatot oldja meg.

A módszer általános

A hátizsák csak egy példa.

Nézzünk két másik feladatot, ahol ugyanez a gondolatmenet működik:

leghosszabb növekvő részsorozat

maximális összegű intervallum

Leghosszabb növekvő részsorozat

INPUT: $A[1 : n]$ különböző egész számokból álló tömb

OUTPUT: egy maximális méretű részsorozat hossza, aminek az elemei növekszenek

Példa: a

$$7,\ 5,\ 2,\ 8,\ 9,\ 3,\ 1,\ 6,\ 4$$

tömbben egy maximális hosszú növekvő részsorozat a 7, 8, 9. Négy hosszú növekvő részsorozat nincs.

A részfeladat és a rekurzió

Definíció

$L[i] :=$ a leghosszabb $A[i]$-re végződő részsorozat hossza.

Állítás

$$L[i] = 1 + \max\Big\{ L[k] \;\Big|\; k < i \;\text{ és }\; A[k] < A[i] \Big\}$$

Állítás

A leghosszabb növekvő részsorozat hossza $\max\Big\{L[i] \;\Big|\; 1 \leq i \leq n\Big\}$.

Példa — a táblázat

i	1	2	3	4	5	6	7	8	9
$A[i]$	9	5	2	8	7	3	1	6	4
$L[i]$									
előző$[i]$									

előző$[i]$ — a leghosszabb $A[i]$-re végződő részsorozat utolsó előtti elemének indexe

Az első három elem

i	1	2	3	4	5	6	7	8	9
$A[i]$	9	5	2	8	7	3	1	6	4
$L[i]$	1	1	1						
előző$[i]$	–	–	–						

A 9, az 5 és a 2 elé nincs náluk kisebb elem,
így mindegyikre $L[i] = 1$.

A negyedik elem

i	1	2	3	4	5	6	7	8	9
$A[i]$	9	5	2	8	7	3	1	6	4
$L[i]$	1	1	1	2					
előző$[i]$	–	–	–	2					

$$L[4] = 1 + \max\{\, L[2],\ L[3] \,\} = 2$$

A kész táblázat

i	1	2	3	4	5	6	7	8	9
$A[i]$	9	5	2	8	7	3	1	6	4
$L[i]$	1	1	1	2	2	2	1	3	3
előző$[i]$	–	–	–	2	2	3	–	6	6

A leghosszabb növekvő részsorozat hossza 3.

Az előző[ ] tömbből visszafejtve: $i = 8 \to 6 \to 3$, azaz 2, 3, 6.

Maximális összegű intervallum

INPUT: $A[1:n]$ egész számokból álló tömb
OUTPUT: egy maximális összegű $A[i..j]$ résztömb összege

- Ha minden $A[i] \geq 0$, könnyű: az egész tömb a maximális.
- Példa: $-2, 1, -3, 4, -1, 2, 1, -5, 4$ esetén 6 a maximális összeg.
- Ötlet: mikor érdemes $A[i]$-t hozzávenni az előző intervallumhoz, és mikor nem?

A rekurzió és a bizonyítás

Definíció

$L[i] :=$ a legnagyobb összegű, $A[i]$-re végződő intervallum összege.

Állítás

$$L[i] = \max\Big\{\, A[i];\quad L[i-1] + A[i] \,\Big\}$$

Bizonyítás. Ha az $A[i]$-re végződő maximális összegű intervallum 1 hosszú, akkor az összeg $A[i]$. Különben az $A[i-1]$-re végződő maximálishoz vesszük hozzá $A[i]$-t. □

Példa

i	1	2	3	4	5	6	7	8	9
$A[i]$	-2	1	-3	4	-1	2	1	-5	4
$L[i]$	-2	1	-2	4	3	5	6	1	5

$$\max\{-2, 1, -2, 4, 3, 5, 6, 1, 5\} = 6$$

Lépésszám

A tábla kitöltése: $\mathcal{O}(n)$

A maximum kikeresése: $\mathcal{O}(n)$

összesen: $\mathcal{O}(n)$

(A teljes kipróbálás módszere $\mathcal{O}(n^2)$.)

Mohó algoritmusok

Alapelv

A részproblémák megoldásakor mindig az aktuálisan legjobbnak tűnő részeredményt választja.

- Hátizsáknál: rendezzük a tárgyakat valamilyen szempont szerint (súly, érték, vagy érték/súly arány), majd sorban rakjuk be, ami még befér.
- Futásideje nagyon jó, gyakran lineáris (a rendezés után).
- De: a 0-1 hátizsákra nem ad mindig optimális megoldást.

A mohó módszer nem optimális — ellenpélda

i	1	2	3
w_i	3	2	2
p_i	5	3	3
p_i/w_i	1,67	1,50	1,50

$W_{\max} = 4$

Mohó (arány szerint)

Berakja az 1. tárgyat ($w = 3$).
A 2. már nem fér be.
érték: 5

Optimális

A 2. és a 3. tárgy ($w = 2 + 2$).

érték: 6

Töredékes hátizsák — itt a mohó optimális

Ha egy tárgynak a részét is berakhatjuk (*fractional knapsack*), akkor az érték/súly arány szerint csökkenő sorrendben haladó mohó algoritmus bizonyítottan optimális.

Az előző példán: 1. tárgy egészben (5), majd a 2. tárgy fele (1,5)

$\Rightarrow$ 6,5 (a 0-1 optimum 6 volt)

Miért működik?

Csereérvvel: ha egy optimális megoldásban van olyan alacsonyabb arányú tárgyrész, amit magasabb arányúra cserélhetnénk, a csere nem rontana — így a mohó választás optimális megoldásba beilleszthető.

Polinomiális-e a $\mathcal{O}(N \cdot W_{\max})$?

Vigyázat!

A $\mathcal{O}(N \cdot W_{\max})$ lépésszám nem polinomiális a bemenet *méretében*.

- A bemenet hossza $\approx \log_2 W_{\max}$ bit, hiszen a $W_{\max}$-ot számjegyekkel írjuk le.
- A lépésszám viszont magával a $W_{\max}$ értékkel nő — ami ebben exponenciális.
- $W_{\max} = 2^{64}$ esetén a tábla kezelhetetlen — pedig a bemenet csak néhány bájt.

Az ilyen algoritmus pszeudo-polinomiális.

A hátizsákprobléma nehézsége

- A 0-1 hátizsák döntési változata NP-teljes — nem ismert rá polinomiális algoritmus.
- Egész súlyok és értékek esetén gyengén NP-teljes: épp ezért létezik rá pszeudo-polinomiális DP-megoldás.
- Létezik rá FPTAS: tetszőleges $\varepsilon > 0$ mellett polinomiális időben kapható olyan megoldás, amely az optimum $(1 - \varepsilon)$-szorosát eléri.

Tehát: pontosan nehéz, de tetszőlegesen jól közelíthető.

A négy módszer összevetése

Módszer	lépésszám	tárigény	optimális?
nyers erő	$\mathcal{O}(2^N)$	$\mathcal{O}(N)$	igen
oszd meg és uralkodj	$\mathcal{O}(2^N)$	$\mathcal{O}(N)$	igen
feljegyzéses	$\mathcal{O}(NW_{\max})$	$\mathcal{O}(NW_{\max})$	igen
dinamikus prog.	$\mathcal{O}(NW_{\max})$	$\mathcal{O}(NW_{\max})$	igen
mohó	$\mathcal{O}(N \log N)$	$\mathcal{O}(N)$	nem

A DP tárigénye $\mathcal{O}(W_{\max})$-re csökkenthető, ha csak az optimum *értéke* kell.

Dinamikus programozás — előnyök, hátrányok

Előnyei

- Futásideje általában kedvező, nincs többszöri számítás.
- Nem rekurzív, így nincs veremkorlát.
- Tárigénye és lépésszáma előre tudható.

Hátrányai

- Az alulról felfelé építkezés miatt néha felesleges részproblémákat is megold.
- Tárigénye magasabb lehet a többi módszerénél.

Összefoglalás

- A dinamikus programozás alulról felfelé tölt ki egy táblát, így minden részproblémát pontosan egyszer old meg.
- Két előfeltétel: optimális részstruktúra és átfedő részfeladatok.
- A hátizsákra: $F_{t,h} = \max\{F_{t-1,h},\ F_{t-1,h-w_t} + p_t\}$, lépésszám $\mathcal{O}(NW_{\max})$.
- A tábla nemcsak az optimum *értékét* adja: belőle visszafejthető maga a pakolás is.
- Ez pszeudo-polinomiális — a feladat NP-nehéz marad.
- A mohó módszer gyorsabb, de a 0-1 hátizsákra nem optimális.

Irodalom

[1] Szénási Sándor: *Optimalizációs stratégiák 1.* Szoftvertervezés és -fejlesztés II., Óbudai Egyetem NIK.

[2] Szénási Sándor: *Algoritmusok, adatszerkezetek II.* Óbudai Egyetem, 2014.

[3] Cormen, Leiserson, Rivest, Stein: *Új algoritmusok* (Introduction to Algorithms). Scolar Kiadó.

[4] Rónyai, Ivanyos, Szabó: *Algoritmusok*. Typotex, 2005.

[5] R. Bellman: *Eye of the Hurricane: An Autobiography*. World Scientific, 1984. (a névadás története)

[6] Kellerer, Pferschy, Pisinger: *Knapsack Problems*. Springer, 2004.